

# Chapter 10: Recursion & Sorting



**Dr. Sirasit Lochanachit**

# Outline



## Recursion

- Components of Recursion
- Summation
- Exponentiation
- Factorial

## Sorting (Cont.)

- Merge Sort

# What is Recursion?



- It breaks down a large problem into smaller subproblems that are simpler to solve<sup>[1]</sup>.
- A function that is calling itself one or more times<sup>[2]</sup>.

[1] Bradley N. Miller, David L. Ranum, Problem Solving with Algorithms and Data Structures using Python, 2011

[2] Michael T. Goodrich et al., Data Structures and Algorithms in Python, 2013

# What is Recursion?



# Recursion



Real-life example:

- Fractal patterns in art and nature
- **Matryoshka doll**
  - Also known as Russian doll or nested doll
  - Set of wooden dolls where smaller dolls are placed inside another.
  - A symbol of motherhood and fertility.



Image Source: <https://images.theconversation.com/files/163337/original/image-20170330-4592-1n4ji0f.jpg>, [https://cdn.pixabay.com/photo/2017/06/13/13/45/matryoshka-2398918\\_960\\_720.jpg](https://cdn.pixabay.com/photo/2017/06/13/13/45/matryoshka-2398918_960_720.jpg),  
<https://tf-cmsv2-smithsonianmag-media.s3.amazonaws.com/filer/1d/ff/1dffc703-583f-43d9-a645-96870f5a1d7b/ice-queen.jpg>

# When to use recursion?



- For problems that contain smaller instances of the same problem.
- To repeat a computer program, **while-loop** and **for-loop** can be used.
- Alternatively, **recursion** repeats by calling a function itself one or more times.

# Components of Recursion



- Base Case: The smallest subproblem that can be solved easily.
- Recursive Case: A subproblem that reduces the size of the input and move toward the base case

# Summation



- Summing list elements recursively.

# Summation



- Summing list elements recursively.

**Time Complexity:  $O(?)$**

```
def getSum(data):  
    """ Return the sum of the first  
    n numbers of sequence data."""  
    totalSum = 0  
    for i in data:  
        totalSum = totalSum + i  
    return totalSum
```

```
def getSum(data):  
    """ Return the sum of the first n  
    numbers of sequence data."""  
    if len(data)==0:  
        return 0  
    else:  
        return data[0]+  
        getSum(data[1:])
```

# Power



**Power** function takes 2 numbers as input and return the value of *base* to the power of *exp*.

$$\text{power}(\text{base}, \text{exp}) = \begin{cases} 1 & \text{if } \text{exp} = 0 \\ \text{base} * \text{power}(\text{base}, \text{exp}-1) & \text{otherwise} \end{cases}$$

```
def getPower(base,exp):  
    """ Return the multiplciation of base with exp times. """  
    if exp==0:  
        return 1  
    else:  
        return base*getPower(base, exp-1)
```

# Factorial



**Factorial** function takes a number as an input and returns the factorial of that number

- $n!$  = Product of positive integers from 1 to  $n$
- If  $n = 0$ , then  $n! = 1$  by convention

factorial(1)

factorial(2)

factorial(3)

factorial(4)

factorial(5)

# Factorial



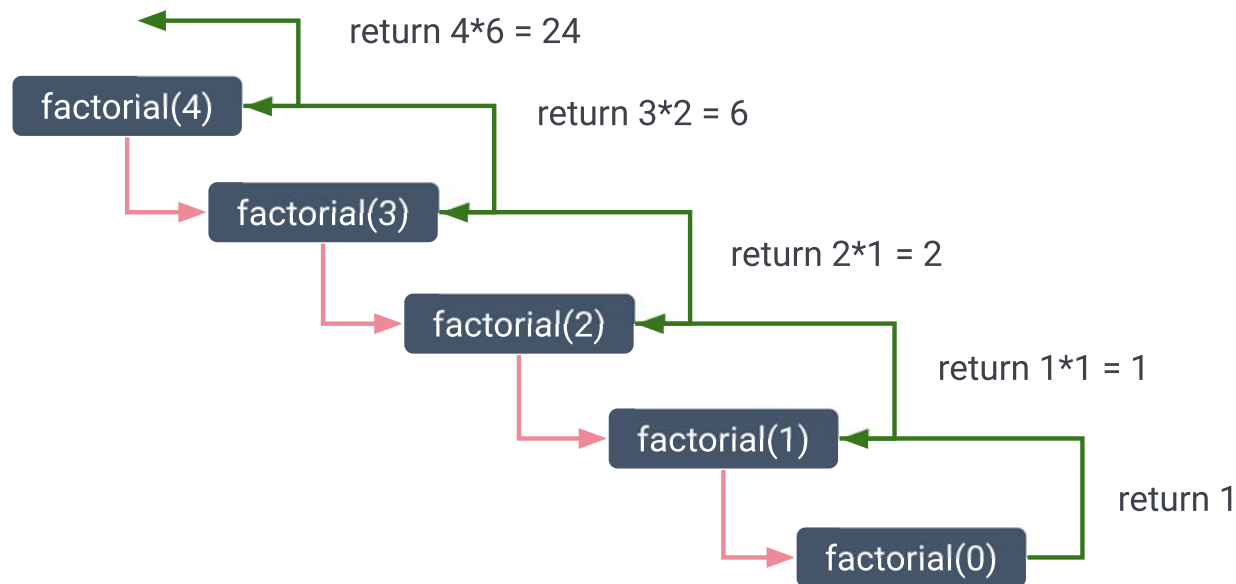
**Factorial** function takes a number as an input and returns the factorial of that number

- $n!$  = Product of positive integers from 1 to  $n$
- If  $n = 0$ , then  $n! = 1$  by convention

# Recursion



- For recursive, repetition is conducted by repeatedly invoke the function call.



# Recap

- Components:
  - Base case and Recursive case
- Examples
  - Summation
  - Power
  - Factorial
  - Binary search
- One rule for recursion: there must be a stop condition.



# Recursion



## Loop

```
# Factorial loop version
def factorial1(n):
    if n == 0:
        return 1
    else:
        total = 1
        for num in range(2,n+1):
            total *= num
        return total
```

**Time Complexity:  $O(?)$**

## Recursive

```
# Factorial recursive
version
def factorial2(n):
    if n == 0:
        return 1
    else:
        return
n*factorial2(n-1)
```

**Time Complexity:  $O(?)$**

06036120 Data Structures and Algorithms Principles

# Divide and Conquer Strategy



3 Steps<sup>[2]</sup>:

1. **Divide**: Divide the input data into two or more disjoint subsets.
2. **Recur**: Recursively solve the subproblems associated with the subsets.
3. **Conquer**: Take the solutions to the subproblems and “merge” them into a solution to the original problem.

# Merge Sort



**Merge sort** is a recursive sorting technique based on divide and conquer technique by continually splits a list into equal halves<sup>[1]</sup>.

**Base case:** If the list is empty or has only one item, it is sorted.

**Recursive case:** If the list has  $> 1$  item, split the list and recursively merge sort on both halves.

After the two halves are sorted, then a merge is invoked, combining them together into a single sorted new list.

[1] Bradley N. Miller, David L. Ranum, Problem Solving with Algorithms and Data Structures using Python, 2011

# Merge Sort Real-life Demo



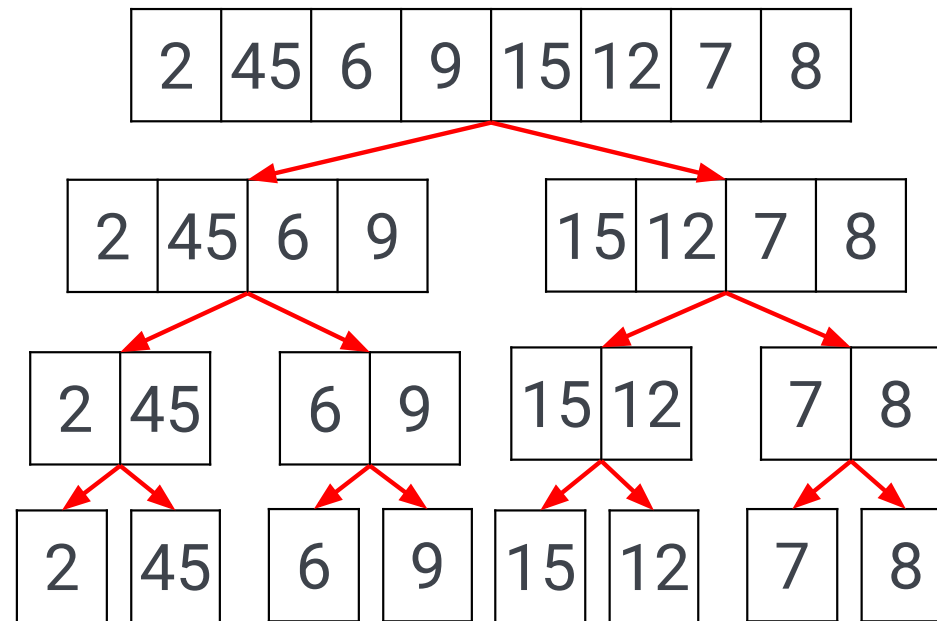
Merge-sort with Transylvanian-saxon (German) folk dance

# Merge Sort Example



|   |    |   |   |    |    |   |   |
|---|----|---|---|----|----|---|---|
| 2 | 45 | 6 | 9 | 15 | 12 | 7 | 8 |
|---|----|---|---|----|----|---|---|

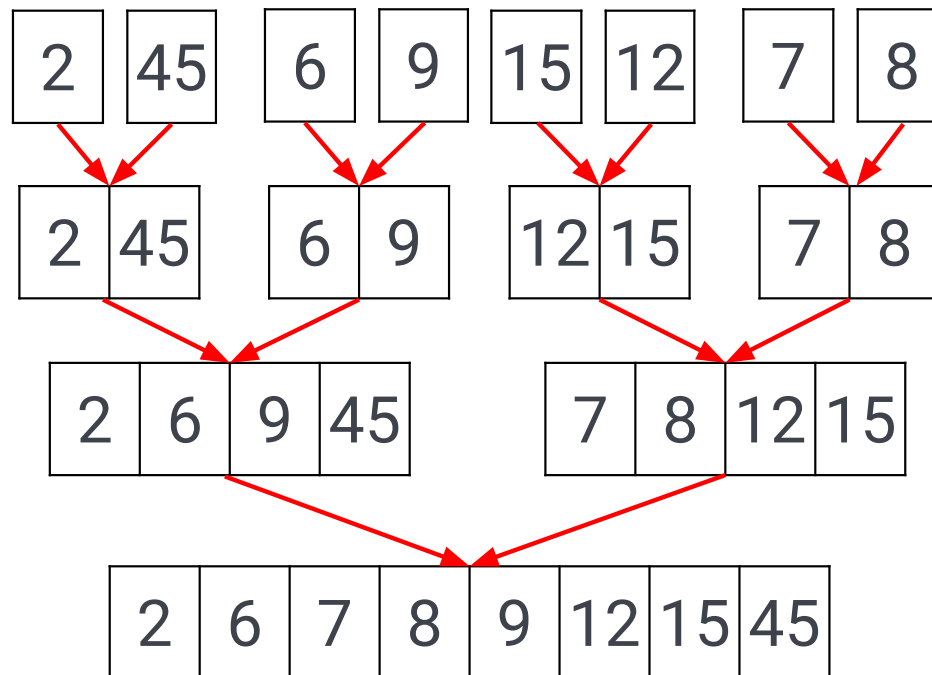
# Merge Sort



# Merge Sort Example



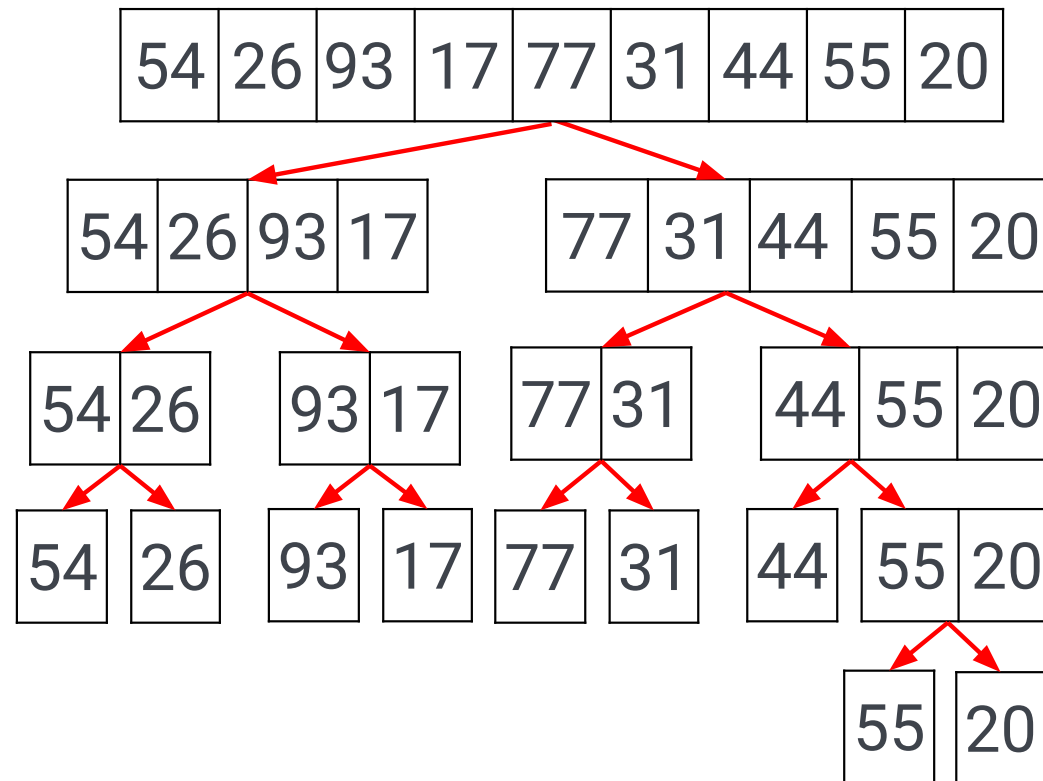
# Merge Sort



# Merge Sort Example



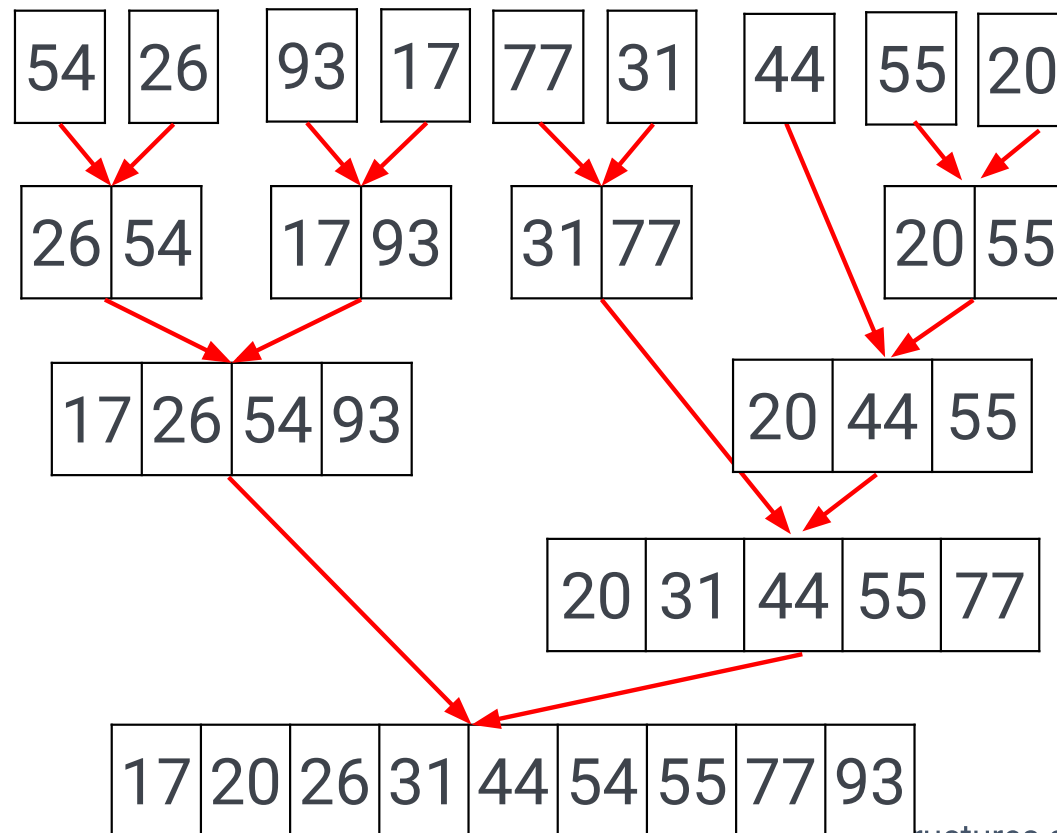
Odd elements



# Merge Sort Example



Odd elements



# Merge Sort Performance



2 Processes<sup>[1]</sup>:

1. Splitting the list into halves
2. Merge operation and sorting

Merge sort requires extra memory space to hold two halves as they are continually splitting.

- This can be a critical factor if the list is large.

Algorithm `mergeSort(list)`:

if `length(list) ≤ 1`:

    return list

else:

    middle = `length(list) / 2`

    divide list into two unsorted sublists, left and right, using middle

`left = mergeSort(left)`

`right = mergeSort(right)`

`result = merge(left, right)`

    return result

end if

End mergeSort